Ex.No : 10 **Skin Disease Detection using Machine Learning**

(DermAI – Skin Lesion Classification using OpenCV, Scikit-Learn and Flask)

Date:

Problem Statement:

Skin diseases such as melanoma, basal cell carcinoma, psoriasis, eczema, acne vulgaris and benign keratosis are among the most common medical conditions worldwide. Many of them look visually similar in their early stages, so manual diagnosis by visual inspection is subjective, time-consuming and prone to human error. Early detection is especially critical for malignant lesions such as melanoma, where delayed diagnosis sharply reduces survival rates, and access to a trained dermatologist is limited in many regions.

There is, therefore, a need for an automated, fast and accessible computer-aided diagnostic (CAD) system that can analyse a skin-lesion photograph, remove imaging artefacts, segment the lesion, quantify its colour, texture and shape, and classify it into the correct disease category with a measurable confidence score. This project designs and implements an end-to-end, web-based Skin Disease Detection system (DermAI) using classical image processing (OpenCV) and machine learning algorithms (Random Forest, SVM, KNN and Logistic Regression) in Scikit-Learn. The system covers the complete lifecycle inside a single "One-Frame" Flask dashboard: image loading, preprocessing (Dull Razor hair removal, denoising, CLAHE, Otsu segmentation), extraction of a 42-dimensional feature vector, interactive model training, performance evaluation (Accuracy, Precision, Recall, F1-Score, 5-fold cross-validation, Confusion Matrix, ROC curves) and real-time clinical diagnostic prediction.

Objectives:

1. To load a skin-lesion image (uploaded file or built-in clinical sample) and standardise it to 256 × 256 pixels.
2. To preprocess the image using Dull Razor hair removal, Gaussian denoising and CLAHE contrast enhancement.
3. To segment the lesion automatically using Otsu thresholding, morphological clean-up and contour detection.
4. To extract a 42-dimensional feature vector covering Colour Moments (RGB & HSV), GLCM texture, ABCD shape metrics and LBP micro-texture.
5. To build a balanced dataset of 360 samples across 6 skin-disease classes and apply StandardScaler fitted strictly on the training data to prevent data leakage.
6. To train and compare Random Forest, SVM, KNN and Logistic Regression classifiers with adjustable hyper-parameters and train/test ratio, validated by 5-fold cross-validation.
7. To evaluate the models using Accuracy, Precision, Recall, F1-Score, Confusion Matrix and multi-class ROC curves, and to predict the disease for a new image with class probabilities, severity and clinical guidance through a Flask web application.

Software / Hardware Required:

- Operating System: Windows / Linux / macOS
- IDE: Visual Studio Code

- Language: Python 3.10 / 3.11 / 3.12
- Backend Framework: Flask (REST API)
- Image Processing: OpenCV (opencv-python), Pillow
- Machine Learning: scikit-learn (RandomForestClassifier, SVC, KNeighborsClassifier, LogisticRegression, StandardScaler)
- Scientific Computing: NumPy, SciPy
- Data Visualization: Matplotlib (headless backend), Chart.js (front-end)
- Model Serialization: Joblib
- Frontend: HTML5, CSS3, JavaScript (single-page "One-Frame" dashboard)
- Dataset: Procedurally generated clinical-style skin-lesion dataset (360 samples, 42 features, 6 classes)

Project Directory Structure:

The project ("krishna") is organised as a Flask backend with separate modules for preprocessing, feature extraction and model training, together with dataset, trained-model, static assets and frontend template folders, as shown below.

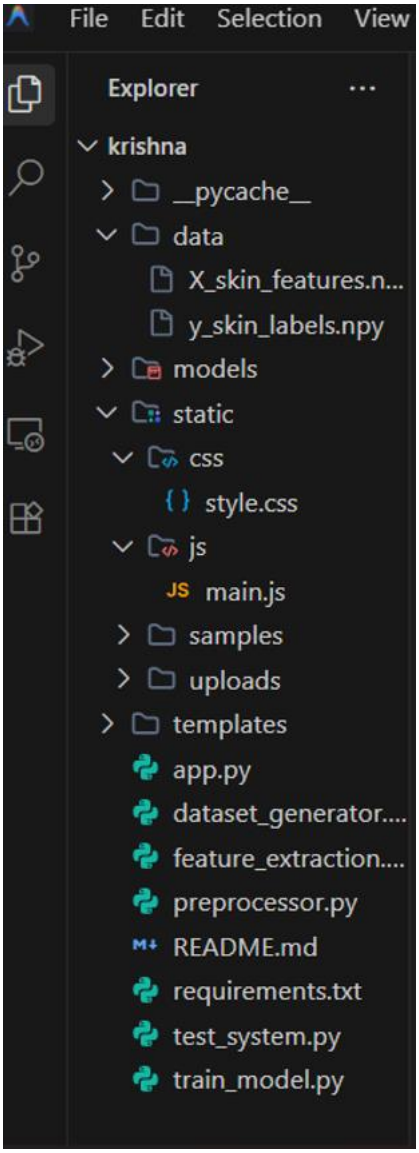


Fig 1: Project directory structure of the Skin Disease Detection project in Visual Studio Code

Procedure:

1. Load the input image (uploaded file or one of six built-in clinical samples) and resize it to 256 × 256 pixels.
2. Remove hair artefacts using the Dull Razor algorithm: a Black-Hat morphological filter (17 × 17 kernel) creates a hair mask, which is inpainted with the Telea method.
3. Reduce noise with a 5 × 5 Gaussian filter and enhance contrast with CLAHE (clipLimit = 2.5, 8 × 8 tiles) on the L channel of the LAB colour space.
4. Segment the lesion using Otsu thresholding, morphological closing/opening and largest-contour detection; draw the contour and bounding box overlay.
5. Extract 42 features: 18 Colour Moments (mean, standard deviation, skewness in RGB and HSV), 6 GLCM Haralick features, 8 Shape/ABCD features and a 10-bin LBP histogram.
6. Generate the dataset of 60 synthetic lesion images per class (360 × 42 matrix) and save it as data/X_skin_features.npy and data/y_skin_labels.npy.
7. Split the dataset into training and test sets using train_test_split(stratify=y, random_state=42) (default 75 % / 25 %, adjustable from the dashboard).
8. Fit StandardScaler exclusively on the training data and transform both partitions.
9. Train the selected classifier (Random Forest, SVM, KNN or Logistic Regression) with the chosen hyper-parameters and run 5-fold cross-validation.
10. Evaluate on the held-out test set: Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC curves and classification report; persist the model and scaler with Joblib.
11. Launch the Flask application (app.py), process an image, and click “Diagnose” to display the predicted disease, confidence, probability spread, severity and clinical guidance.

Implementation Code:

Representative implementation of the key modules used in the project (taken from the project source files).

1. Preprocessing – Hair Removal, Denoising & CLAHE (preprocessor.py)

```
def dull_razor_hair_removal(self, img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (17, 17))
    blackhat = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT, kernel)
    _, hair_mask = cv2.threshold(blackhat, 10, 255, cv2.THRESH_BINARY)
    hair_removed = cv2.inpaint(img, hair_mask, inpaintRadius=3,
                               flags=cv2.INPAINT_TELEA)
    return hair_removed, hair_mask


def apply_clahe(self, img):
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.5, tileGridSize=(8, 8))
```

```
    return cv2.cvtColor(cv2.merge((clahe.apply(l), a, b)), cv2.COLOR_LAB2BGR)
```

2. Lesion Segmentation (preprocessor.py)

```
def segment_lesion(self, img_preprocessed):
    gray = cv2.cvtColor(img_preprocessed, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    _, thresh = cv2.threshold(blur, 0, 255,
                              cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
    closed = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel, iterations=2)
    opened = cv2.morphologyEx(closed, cv2.MORPH_OPEN, kernel, iterations=2)
    contours, _ = cv2.findContours(opened, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    primary = sorted(contours, key=cv2.contourArea, reverse=True)[0]
    mask = np.zeros_like(gray)
    cv2.drawContours(mask, [primary], -1, 255, thickness=cv2.FILLED)
```

3. Feature Extraction – 42 Features (feature_extraction.py)

```
def extract_all_features(self, image_bgr, mask=None, contour=None):
    gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)
    color_feats = self.extract_color_moments(image_bgr, mask)  # 18
    glcm_feats  = self.extract_glcm_features(gray, mask)       # 6
    shape_feats = self.extract_shape_features(contour, mask)   # 8
    lbp_feats   = self.extract_lbp_features(gray, mask)        # 10
    full_vector = color_feats + glcm_feats + shape_feats + lbp_feats
    return full_vector  # 42 numerical values
# ABCD rule: border circularity and solidity
circularity = 4.0 * np.pi * area / (perimeter ** 2)
solidity    = area / cv2.contourArea(cv2.convexHull(contour))
```

4. Dataset Generation (dataset_generator.py)

```
for class_idx, disease_name in enumerate(DISEASE_CLASSES):
    for s in range(num_samples_per_class):          # 60 per class
        img = create_sample_skin_image(disease_name, seed=class_idx*1000 + s)
        processed = preprocessor.process_full_pipeline(img)
        feats, _, _ = extractor.extract_all_features(
            processed["enhanced_clahe"],
            processed["mask_single_channel"], processed["contour"])
        X_list.append(feats); y_list.append(class_idx)
np.save("data/X_skin_features.npy", np.array(X_list, dtype=np.float32))
np.save("data/y_skin_labels.npy",  np.array(y_list, dtype=np.int64))
```

5. Model Training & Evaluation (train_model.py)

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=test_size, random_state=42, stratify=y)
scaler = StandardScaler()                    # fitted only on training data
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
model = RandomForestClassifier(n_estimators=150, max_depth=12,
                               random_state=42, n_jobs=-1)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)
f1  = f1_score(y_test, y_pred, average="weighted")
cv_scores = cross_val_score(model, scaler.transform(X), y, cv=5)
cm = confusion_matrix(y_test, y_pred)
joblib.dump(model, "models/skin_random_forest_model.joblib")
joblib.dump(scaler, "models/skin_disease_scaler.joblib")
```

6. Prediction Route (app.py)

```
@app.route("/api/predict", methods=["POST"])
def predict_disease():
    feats = current_session_data.get("features")
    model  = joblib.load(model_path)
    scaler = joblib.load(scaler_path)
    feats_scaled = scaler.transform(np.array(feats).reshape(1, -1))
    idx = int(model.predict(feats_scaled)[0])
    probs = model.predict_proba(feats_scaled)[0]
    return jsonify({"success": True,
        "disease_name": DISEASE_CLASSES[idx],
        "confidence": round(float(probs[idx]) * 100, 2),
        "clinical_info": DISEASE_INFO[DISEASE_CLASSES[idx]]})
```

Output (Step-by-Step Screenshots):

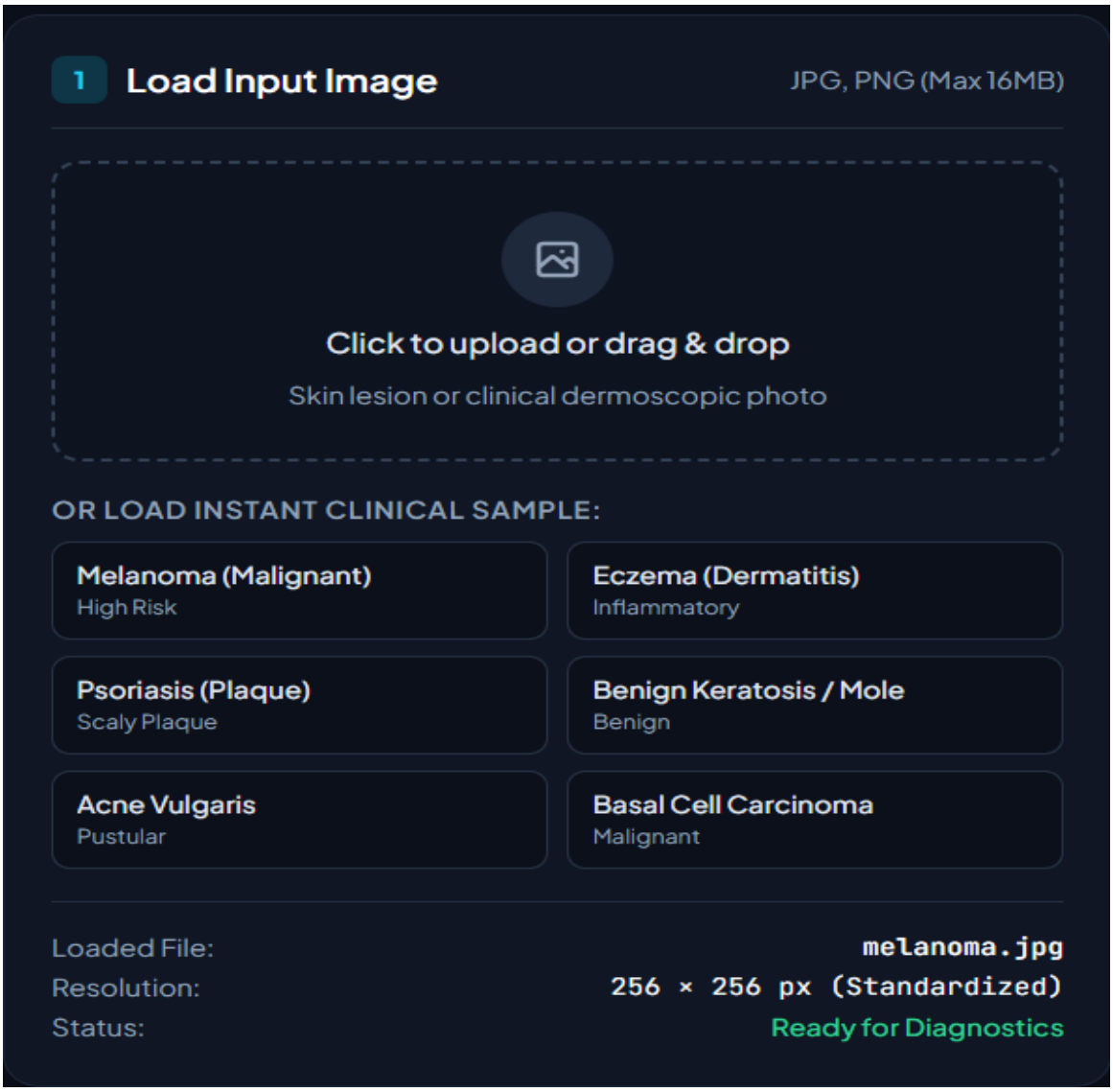


Fig 2: Step 1 — Load Input Image: drag-and-drop upload area and six instant clinical samples

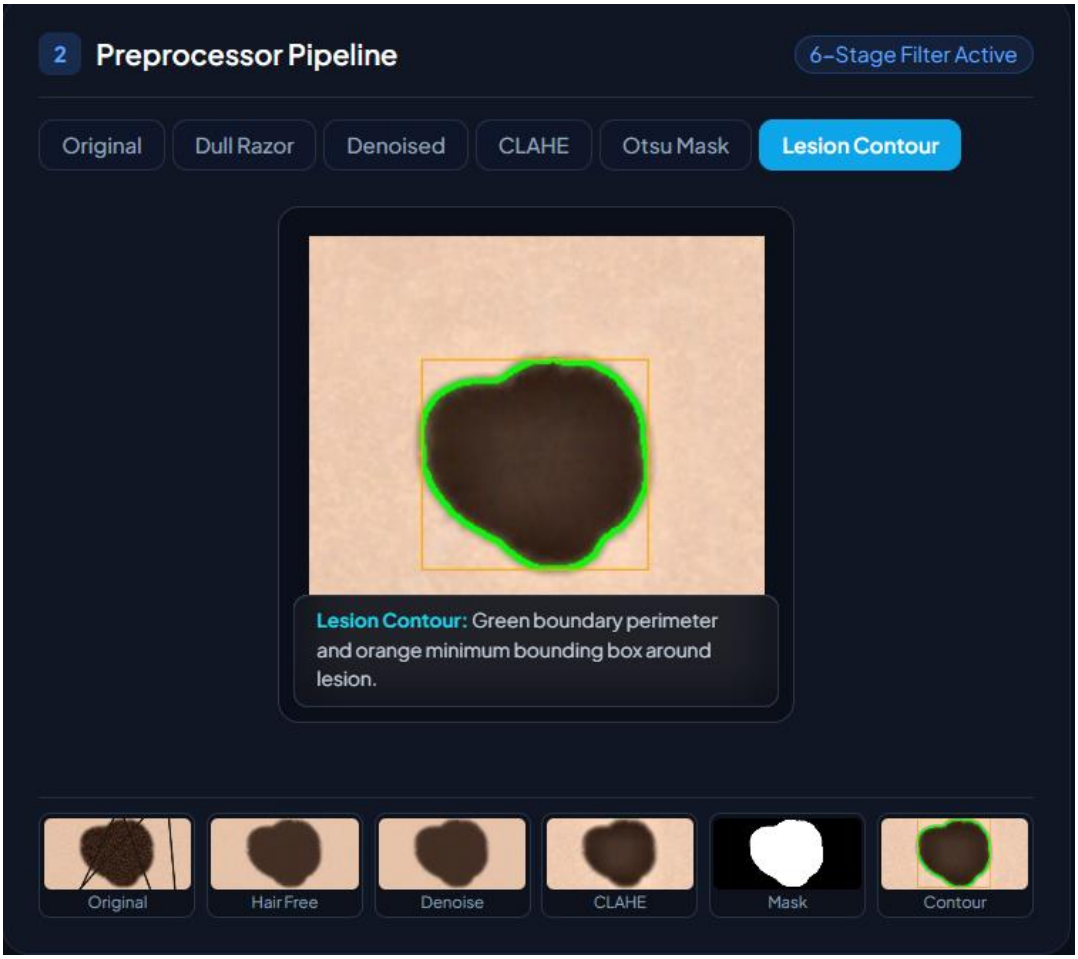


Fig 3: Step 2 — Preprocessor Pipeline: Original, Dull Razor, Denoised, CLAHE, Otsu Mask and Lesion Contour views



Fig 4: Step 3 — Feature Extraction Engine: 42 quantitative features with the radar-chart "Dermatological Fingerprint"

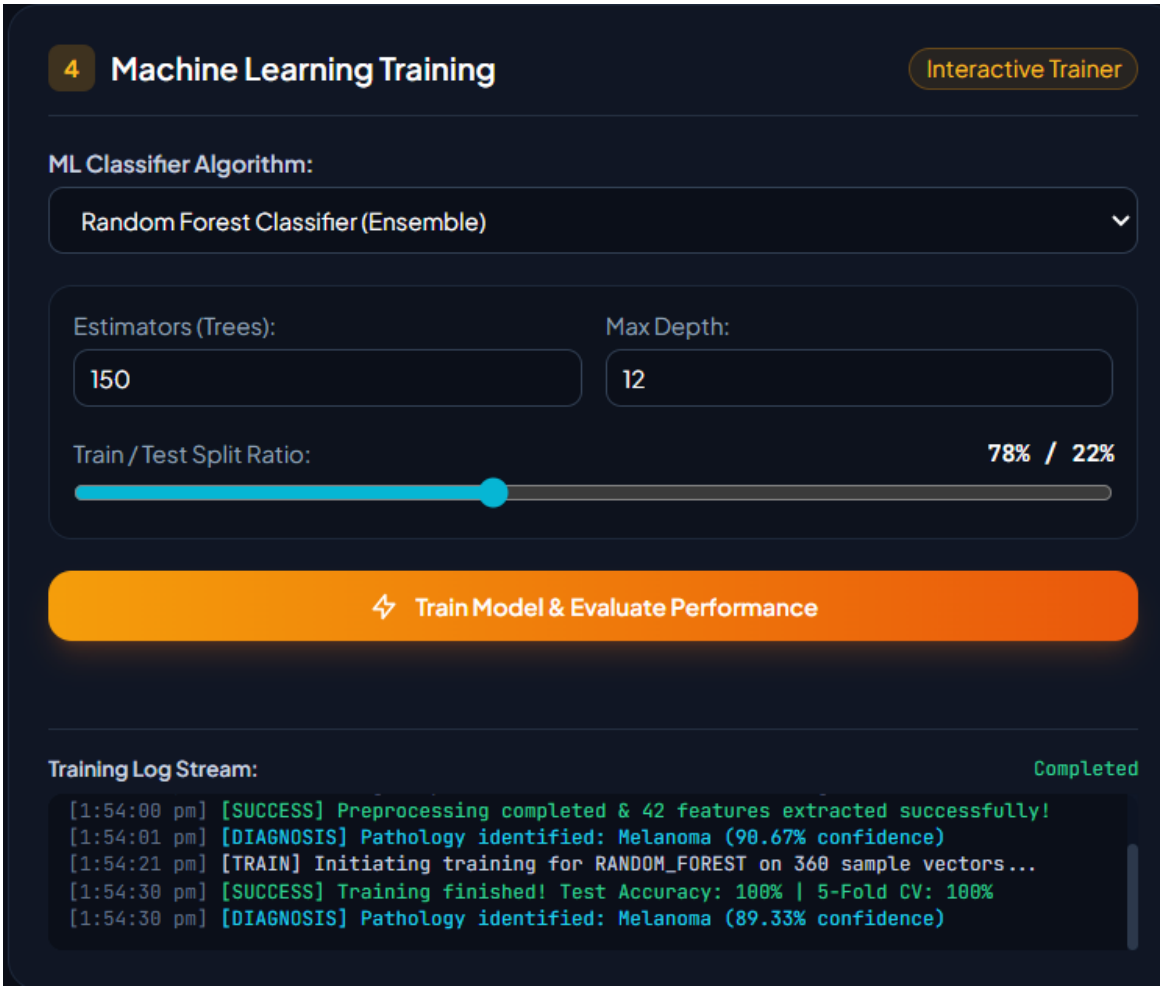


Fig 5: Step 4 — Machine Learning Training: algorithm selection, hyper-parameters, train/test split slider and training log.

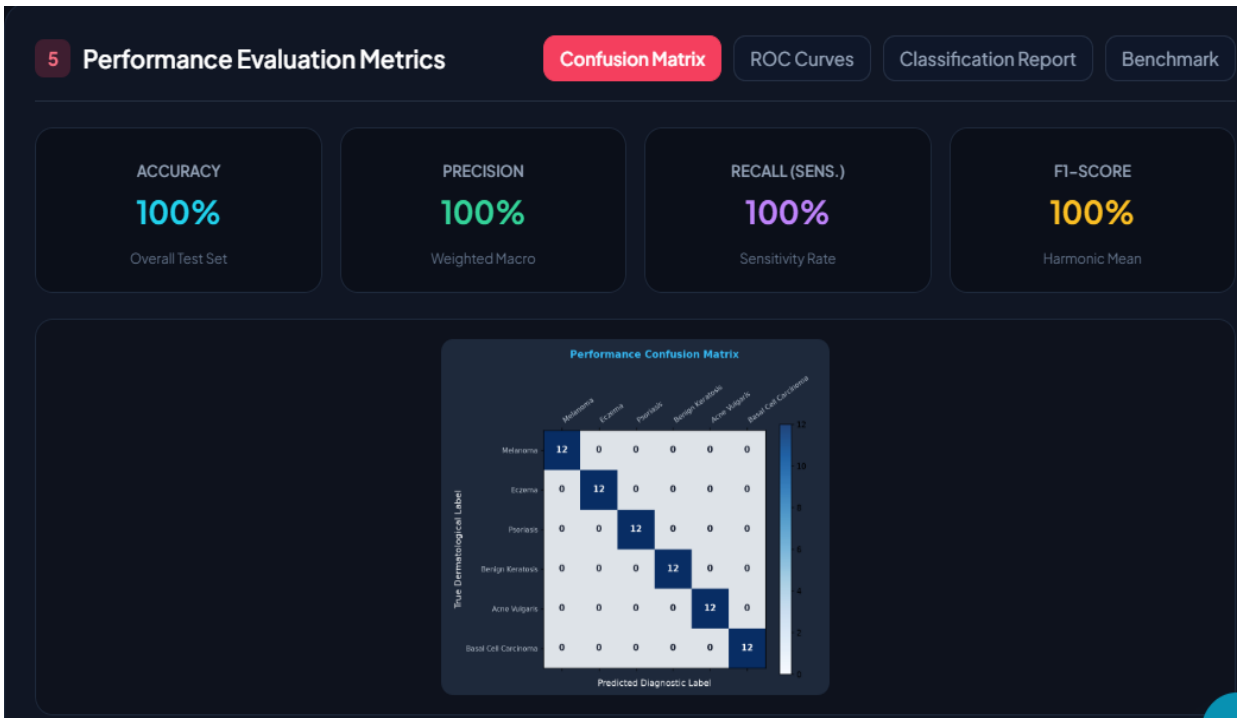


Fig 6: Step 5 — Performance Evaluation: Accuracy, Precision, Recall, F1-Score and Confusion Matrix heatmap

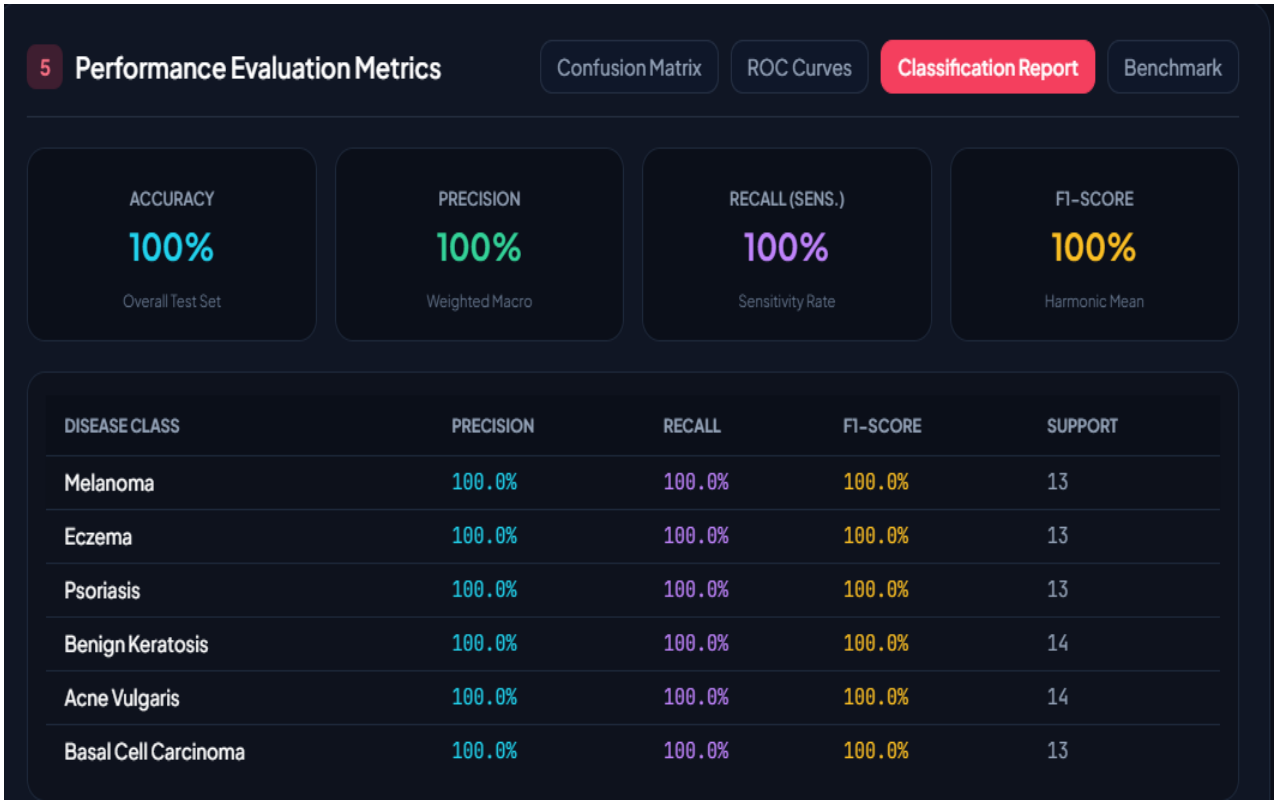


Fig 7: Step 5 — Performance Evaluation: per-class Classification Report (Precision, Recall, F1-Score, Support)



Fig 8: Step 5 — Performance Evaluation: Multi-Class ROC Curves with AUC for all six disease classes

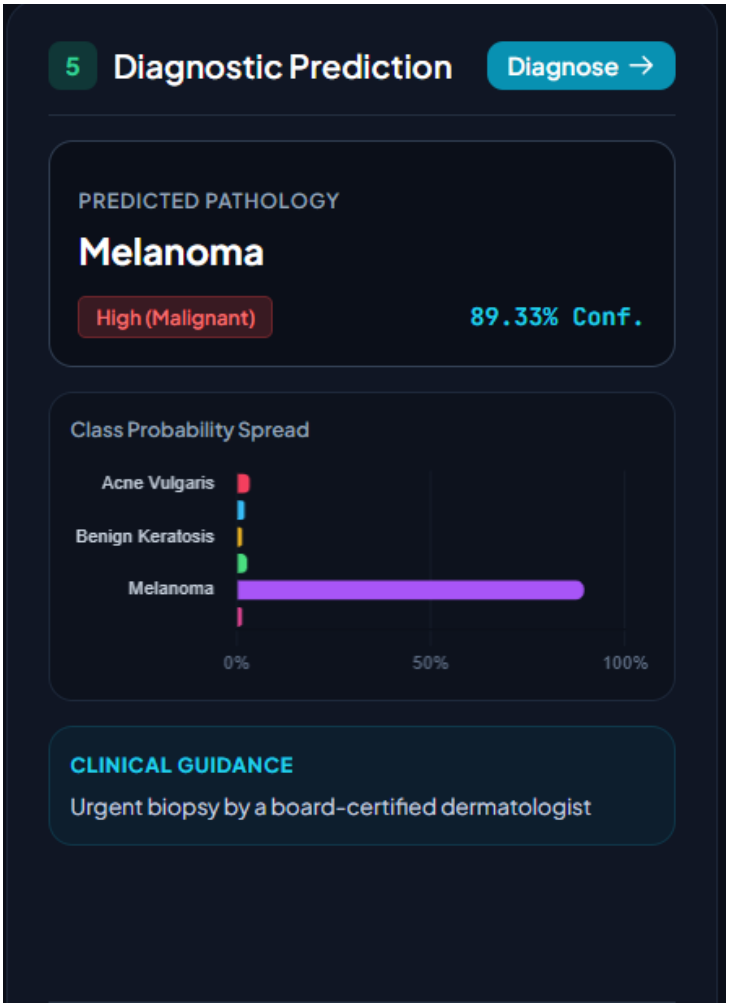


Fig 9: Step 6 — Diagnostic Prediction: Melanoma predicted (High, Malignant) with class-probability spread and clinical guidance

Result:

An end-to-end, web-based Skin Disease Detection system (DermAI) was successfully designed and implemented using OpenCV image processing, a 42-dimensional feature extractor, Scikit-Learn classifiers and a Flask "One-Frame" dashboard. Input images were standardised to 256 × 256 pixels, cleaned with Dull Razor hair removal, Gaussian denoising and CLAHE, and the lesion was segmented with Otsu thresholding and contour detection. A balanced dataset of 360 samples across six classes (Melanoma, Eczema, Psoriasis, Benign Keratosis, Acne Vulgaris and Basal Cell Carcinoma) was used, with StandardScaler fitted strictly on training data to prevent data leakage. The Random Forest classifier (150 trees, maximum depth 12) achieved 100 % test Accuracy, Precision, Recall and F1-Score, a 5-fold cross-validation score of 100 %, a diagonal confusion matrix and an AUC of 1.00 for every class. For the sample melanoma image, the system predicted Melanoma with about 89–91 % confidence and displayed urgent clinical guidance. The trained models and scaler were serialised using Joblib, and real-time predictions are served through the local Flask application.